Semantic Search Engine

Complete 8-Week Roadmap

MAANG/FAANG Level Portfolio Project

B.Tech Final Year - Data Science

Tech Stack: Python | Sentence-BERT | FAISS | BM25 | FastAPI | Streamlit

100% Free Tools | Wikipedia Domain | Cloud Deployment

Table of Contents

1. Project Overview & Goals 3
2. Tech Stack (All Free) 3
3. Project Structure 4
4. Week 1: Foundation & Data Pipeline 5
5. Week 2: Semantic Embeddings & FAISS 6
6. Week 3: Hybrid Search & Fusion 7
7. Week 4: Explainability & Evaluation 8
8. Week 5: FastAPI Backend 9
9. Week 6: Streamlit Dashboard 10
10. Week 7: Documentation & Testing 11
11. Week 8: Deployment & Portfolio 12
12. Key Metrics & Targets 13
13. Interview Questions 14
14. Resources & References 15

1. Project Overview & Goals

The Problem

Traditional keyword search fails to understand user intent. Searching 'how to fix sad plant' should find articles about 'reviving dying houseplants' - but BM25 and TF-IDF miss this semantic connection. This project builds a search engine that understands MEANING, not just keywords.

What You Will Build

- A hybrid search engine combining BM25 (keyword) + Sentence-BERT (semantic) + FAISS (vector search)
- Search across 200,000+ Wikipedia articles with sub-100ms query latency
- Explainable results showing WHY each document was retrieved
- Rigorous evaluation with IR metrics (MRR, NDCG, Recall@K)
- Production-grade FastAPI backend with REST API
- Beautiful Streamlit dashboard deployed on Streamlit Cloud
- Complete documentation, tests, and portfolio materials

Why This Is MAANG-Level

- Hybrid search is exactly what Google, Amazon, and Netflix use in production
- Shows system thinking: data pipeline, ML models, API design, frontend, deployment
- Demonstrates evaluation rigor: proper IR metrics, error analysis, benchmarking
- Production-quality code: tests, documentation, type hints, linting
- End-to-end ownership: from raw data to deployed product

2. Tech Stack (All Free)

Component	Tool	Why
Language	Python 3.10+	Industry standard for ML
Embeddings	sentence-transformers	Best open-source embeddings
Vector Index	FAISS (CPU)	Facebook's vector search - blazing fast
BM25	rank_bm25	Production-proven keyword search
Dataset	HuggingFace datasets	Free Wikipedia corpus (200K docs)
API Backend	FastAPI	Modern, fast, auto-documentation
Web App	Streamlit	Rapid ML app development
Tracking	Weights & Biases	Free experiment tracking
Testing	pytest	Industry standard testing
Code Quality	ruff + black	Fast linting and formatting
GPU	Google Colab	Free GPU for embeddings
Hosting	Streamlit Cloud	Free deployment with custom URL

Key Insight

Every single tool in this stack is free and open-source. No API keys, no subscriptions, no credit card required. You can build and deploy this entire project for $0.

3. Project Structure

```
semantic-search-engine/
|-- README.md                    # Comprehensive project docs
|-- pyproject.toml               # Project configuration
|-- requirements.txt             # Dependencies
|-- .gitignore
|
|-- configs/
|   |-- default.yaml             # Default configuration
|   +-- production.yaml          # Production config
|
|-- src/
|   |-- __init__.py
|   |-- data/
|   |   |-- __init__.py
|   |   |-- downloader.py        # Wikipedia dataset download
|   |   |-- preprocessor.py      # Text cleaning pipeline
|   |   +-- loader.py            # Data loading utilities
|   |
|   |-- embeddings/
|   |   |-- __init__.py
|   |   |-- encoder.py           # Sentence-BERT encoding
|   |   +-- cache.py             # Embedding caching
|   |
|   |-- index/
|   |   |-- __init__.py
|   |   |-- bm25_index.py        # BM25 keyword index
|   |   |-- faiss_index.py       # FAISS vector index
|   |   +-- hybrid_index.py      # Combined hybrid index
|   |
|   |-- search/
|   |   |-- __init__.py
|   |   |-- bm25_searcher.py     # BM25 search engine
|   |   |-- semantic_searcher.py # Semantic search engine
|   |   |-- hybrid_searcher.py   # Hybrid search (RRF fusion)
|   |   +-- query_processor.py   # Query understanding
|   |
|   |-- evaluation/
|   |   |-- __init__.py
|   |   |-- metrics.py           # IR metrics (NDCG, MRR, etc)
|   |   |-- dataset.py           # Evaluation dataset builder
|   |   +-- evaluate.py          # Evaluation pipeline
|   |
|   |-- api/
|   |   |-- __init__.py
|   |   |-- main.py              # FastAPI application
|   |   |-- models.py            # Pydantic request/response models
|   |   +-- routes.py            # API route handlers
|   |
|   +-- utils/
|       |-- __init__.py
|       |-- config.py            # YAML config management
|       |-- logging.py           # Rich logging setup
|       +-- helpers.py           # Utility functions
|
|-- app/
|   +-- streamlit_app.py         # Streamlit dashboard
|
|-- notebooks/
|   |-- 01_data_exploration.ipynb
|   |-- 02_embedding_analysis.ipynb
|   +-- 03_evaluation.ipynb
|
|-- tests/
|   |-- conftest.py              # Shared test fixtures
|   |-- unit/
|   |   |-- test_preprocessor.py
|   |   |-- test_encoder.py
|   |   +-- test_metrics.py
|   |-- integration/
```

```
|   |   |-- test_search_pipeline.py
|   |   +-- test_api.py
|   +-- performance/
|       +-- test_latency.py
|
|-- scripts/
|   |-- build_index.py           # Build search index
|   |-- run_evaluation.py        # Run full evaluation
|   +-- deploy.py                # Deployment helper
|
|-- docs/
|   |-- architecture.md          # System architecture
|   |-- api.md                   # API documentation
|   +-- benchmarks.md            # Performance benchmarks
|
+-- data/
    |-- raw/                     # Raw Wikipedia data
    |-- processed/               # Cleaned & tokenized
    |-- index/                   # Built search indices
    +-- evaluation/              # Labeled eval queries
```

WEEK 1: Foundation & Data Pipeline

Goal: Set up project, build data ingestion, create BM25 baseline

Day 1-2: Project Setup

> Create full project directory structure
> Set up pyproject.toml with all dependencies (torch, sentence-transformers, faiss, etc.)
> Initialize git repository with .gitignore for Python/ML projects
> Create Python virtual environment and install dependencies
> Configure ruff (linting) + black (formatting) for code quality
> Create README.md skeleton with badges and project description
> Set up config management with YAML files + environment variable overrides

Day 3-4: Data Ingestion Pipeline

> Download Wikipedia dataset using HuggingFace datasets library (200K articles)
> Build text preprocessing pipeline: HTML cleaning, Unicode normalization
> Implement sentence splitting with NLTK or spaCy
> Tokenization and stopword removal
> Store processed documents as JSON Lines (efficient for large datasets)
> Add data validation: check for empty docs, encoding issues, duplicates
> Log processing statistics: docs/sec, total size, filtered count

Day 5-7: BM25 Baseline

> Implement BM25 index using rank_bm25 library
> Tokenize documents and build inverted index
> Implement query processing: tokenize, remove stopwords, stem
> Return top-K results with BM25 scores
> Create evaluation harness with 50 hand-labeled test queries
> Measure baseline metrics: Recall@10, MRR, NDCG@10
> Document baseline performance as reference for improvement

Week 1 Deliverables:

[x] Complete project structure with all configuration files
[x] Working data pipeline processing 200K Wikipedia documents
[x] BM25 search engine returning relevant results
[x] Baseline metrics documented (Recall@10, MRR, NDCG)

WEEK 2: Semantic Embeddings & FAISS

Goal: Build vector embeddings and FAISS index for semantic search

Day 1-2: Sentence-BERT Embeddings

> Load all-MiniLM-L6-v2 model from sentence-transformers (384-dim, 80MB)
> Build batch embedding pipeline with configurable batch_size=256
> Process 200K documents in batches, track progress with tqdm
> Implement embedding caching: save to disk as .npy files to avoid re-computation
> Benchmark embedding speed: target 1000+ docs/second on GPU
> Store document IDs alongside embeddings for result mapping

Day 3-4: FAISS Index

> Learn FAISS index types: IndexFlatL2 (exact), IndexIVFFlat (approx), IndexHNSW (graph)
> Build IndexFlatL2 as baseline (exact search, O(n) but fast for 200K)
> Implement save/load for FAISS indices (faiss.write_index / read_index)
> Benchmark query latency: target < 100ms for top-10 retrieval
> Test with various query types: single word, phrase, question, typo

Day 5-7: Semantic Search Pipeline

> Build end-to-end pipeline: Query -> Encode -> FAISS Search -> Map IDs -> Return Results
> Compare BM25 vs Semantic search on 50 test queries side-by-side
> Analyze where semantic search wins: synonyms, paraphrases, concept matching
> Analyze where BM25 wins: exact matches, proper nouns, specific terms
> Create t-SNE/UMAP visualization of embedding space (2D scatter plot)
> Document trade-offs and choose when to use each method

Week 2 Deliverables:

- [x] 200K document embeddings computed and cached to disk
- [x] FAISS index built with < 100ms query latency
- [x] Semantic search pipeline working end-to-end
- [x] Comparative analysis: BM25 vs Semantic search with examples

WEEK 3: Hybrid Search & Fusion

Goal: Combine BM25 + Semantic search using Reciprocal Rank Fusion

Day 1-2: Reciprocal Rank Fusion (RRF)

> Implement RRF: score(d) = sum(1 / (k + rank_i(d))) for each ranker
> Default k=60 parameter, test k in [30, 60, 90, 120]
> Benchmark hybrid vs BM25-only vs Semantic-only on evaluation set
> Visualize rank correlation between BM25 and semantic results

Day 3-4: Query Understanding

> Spell correction using pyspellchecker (fix 'machin lerning' -> 'machine learning')
> Synonym expansion using WordNet (expand 'car' to include 'automobile', 'vehicle')
> Query tokenization: handle multi-word phrases, acronyms, proper nouns
> Optional: T5-based query rewriting for complex queries

Day 5-7: Advanced Features

> Pseudo-relevance feedback: expand query using top-K results terms
> Document filtering by metadata (title contains, text length, etc.)
> Query suggestion: given partial query, suggest completions
> Search result clustering: group results by topic/theme
> A/B comparison tool: show BM25 vs Semantic vs Hybrid side by side

Week 3 Deliverables:

- [x] Hybrid search with RRF beating both individual methods
- [x] Query understanding pipeline with spell correction + expansion
- [x] Advanced search features (filtering, clustering, suggestions)

WEEK 4: Explainability & Evaluation

Goal: Make results interpretable + build rigorous evaluation pipeline

Day 1-2: Explainability

> Highlight matching terms in search results (BM25 keyword matches)
> Show semantic similarity scores for each result
> Create 'Why this result?' explanation panel
> Visualize attention weights for transformer-based matches
> Score breakdown: BM25 contribution vs Semantic contribution

Day 3-4: Evaluation Pipeline

> Create labeled evaluation dataset: 200 queries with 5+ relevant docs each
> Implement Precision@K and Recall@K metrics
> Implement Mean Reciprocal Rank (MRR)
> Implement Normalized Discounted Cumulative Gain (NDCG@K)
> Implement Mean Average Precision (MAP)
> Run evaluation: BM25 vs Semantic vs Hybrid across all metrics

Day 5-7: Error Analysis

> Identify failure modes: no-match queries, ambiguous queries, jargon
> Categorize errors: vocabulary mismatch, synonym gap, domain specificity
> Create confusion matrix of search quality by query type
> Document top-10 failure cases with root cause analysis
> Plan improvements: which errors are fixable, which are inherent limits

Week 4 Deliverables:

- [x] Explainable search results with highlighted terms and score breakdown
- [x] Comprehensive evaluation report with all IR metrics
- [x] Error analysis document with categorized failure modes

WEEK 5: FastAPI Backend

Goal: Build production-grade REST API for the search engine

Day 1-2: API Design

> Design REST API endpoints:
> POST /search - Main search (query, top_k, search_method params)
> GET /health - Health check with index status
> POST /documents - Add new documents to index
> GET /stats - Index statistics (doc count, index size, etc.)
> GET /explain/{doc_id} - Explain why a document was returned
> Create OpenAPI/Swagger documentation with examples

Day 3-4: API Implementation

> Implement FastAPI application with proper project structure
> Create Pydantic models for request/response validation
> Add rate limiting (60 requests/minute per IP)
> Implement in-memory caching for frequent queries (LRU cache)
> Add proper error handling: 400, 404, 429, 500 responses
> Structured logging with request IDs and timing

Day 5-7: Testing & Optimization

> Write unit tests for all API endpoints (pytest + httpx)
> Write integration tests for search pipeline through API
> Load testing with locust: measure throughput and latency under load
> Optimize: batch encoding, index caching, connection pooling
> Optional: Create Docker container for reproducible deployment

Week 5 Deliverables:

- [x] Working FastAPI backend with all endpoints + Swagger docs
- [x] API tests passing with > 80% coverage
- [x] Load test results: throughput (req/sec) and p99 latency

WEEK 6: Streamlit Dashboard

Goal: Build beautiful, interactive search interface

Day 1-2: Core UI

> Create search bar with autocomplete suggestions
> Display results as cards with title, snippet, relevance score
> Highlight matching terms in result snippets
> Add loading spinner during search
> Handle empty results gracefully with suggestions

Day 3-4: Advanced Features

> Sidebar with search filters (method selection, result count)
> Search history panel (store last 20 queries in session state)
> Result comparison view: show BM25 vs Semantic vs Hybrid side by side
> Analytics dashboard: query stats, latency distribution, popular terms
> Index statistics panel: document count, embedding dimensions, index size

Day 5-7: Polish & Optimization

> Dark/Light theme toggle using Streamlit theming
> Responsive layout for mobile and tablet
> About page: project motivation, architecture diagram, how it works
> Demo queries section with pre-loaded examples
> Performance: cache model loading, lazy-load index, session state
> Add Streamlit components for metrics visualization (plotly charts)

Week 6 Deliverables:

[x] Beautiful, polished Streamlit dashboard with all features
[x] Search history, filtering, and comparison views working
[x] About page with architecture explanation

WEEK 7: Documentation & Testing

Goal: Production-quality documentation and comprehensive tests

Day 1-2: Documentation

> Write comprehensive README.md with: overview, architecture, setup, usage
> Create architecture.md with system diagrams (draw.io or mermaid)
> Write API documentation with request/response examples
> Document benchmark results: latency, throughput, accuracy metrics
> Add contributing guidelines and code of conduct

Day 3-4: Testing

> Write unit tests for preprocessor, encoder, metrics modules
> Integration tests: full search pipeline from query to results
> API tests: all endpoints with valid and invalid inputs
> Performance regression tests: latency thresholds
> Target: 80%+ test coverage with pytest-cov

Day 5-7: Code Quality

> Run ruff linter and fix all warnings
> Run black formatter on all Python files
> Add type hints to all function signatures
> Review and refactor any technical debt
> Create CONTRIBUTING.md with development guidelines

Week 7 Deliverables:

- [x] Complete documentation (README, API docs, architecture docs)
- [x] 80%+ test coverage across all modules
- [x] Clean, well-formatted, type-hinted codebase

WEEK 8: Deployment & Portfolio

Goal: Deploy to production and create interview-ready portfolio

Day 1-2: Streamlit Cloud Deployment

> Prepare requirements.txt for Streamlit Cloud
> Handle large files: Git LFS for indices or HuggingFace Hub upload
> Set up Streamlit secrets for any sensitive configuration
> Deploy to share.streamlit.io and test end-to-end
> Fix any deployment issues (memory, timeouts, dependencies)

Day 3-4: Portfolio Materials

> Create 2-3 minute demo video (OBS Studio - free screen recording)
> Write Medium blog post: 'Building a Semantic Search Engine from Scratch'
> Prepare GitHub repository: badges, screenshots, GIFs, clear README
> Create LinkedIn post highlighting technical achievements
> Record architecture walkthrough for interview prep

Day 5-7: Interview Preparation

> Prepare answers for 10+ interview questions (see Section 13)
> Practice 2-minute project pitch for behavioral interviews
> Prepare for technical deep-dives on each component
> Create 'lessons learned' document for interview talking points
> Mock interview with a friend or record yourself explaining

Week 8 Deliverables:

- [x] Live deployment URL you can share in applications
- [x] Professional GitHub repository ready for stars
- [x] Blog post and demo video for portfolio
- [x] Interview prep document with 10+ answered questions

12. Key Metrics & Targets

Metric	Target	Why It Matters
Recall@10	> 0.85	Most relevant docs found in top 10
Mean Reciprocal Rank	> 0.75	First relevant result appears early
NDCG@10	> 0.80	Ranking quality (position-weighted)
Query Latency (p50)	< 50ms	Feels instant to users
Query Latency (p99)	< 100ms	Production-ready speed
Index Build Time	< 30 min	Reasonable for 200K docs
Embedding Speed	> 1000 docs/sec	Efficient batch processing
Test Coverage	> 80%	Code quality and reliability
API Throughput	> 50 req/sec	Handles real traffic

Success Criteria Checklist

- [x] Live, working search engine at a public Streamlit Cloud URL
- [x] GitHub repository with comprehensive README and documentation
- [x] Blog post explaining your approach on Medium
- [x] 2-3 minute demo video for portfolio
- [x] Answers prepared for 10+ technical interview questions
- [x] Evaluation metrics that beat simple baselines by > 15%
- [x] Clean codebase with type hints, tests, and linting

13. Interview Questions to Prepare

Architecture Questions

Q: "Why hybrid search instead of just semantic?"

A: BM25 excels at exact keyword matching (proper nouns, specific terms). Semantic search excels at conceptual similarity (synonyms, paraphrases). Hybrid combines both strengths using Reciprocal Rank Fusion, achieving better recall than either alone.

Q: "How does Reciprocal Rank Fusion work?"

A: RRF combines ranked lists by computing score = sum(1/(k + rank_i)) across all rankers. It's parameter-free (k=60 works well), doesn't need score normalization, and is robust to score distribution differences between methods.

Q: "What is the trade-off between FAISS IndexFlatL2 vs IndexIVFFlat?"

A: FlatL2 is exact (brute force) - O(n) per query but simple. IVFFlat partitions vectors into clusters, searches only nearby clusters - O(n/k) but approximate. For 200K docs, FlatL2 is fast enough. IVFFlat is needed at 10M+ scale.

Q: "How would you scale to 100M documents?"

A: Switch to IVFFlat or HNSW index. Shard across multiple machines. Use approximate nearest neighbor with nprobe parameter. Add caching layer for frequent queries. Consider distributed FAISS with sharding.

System Design Questions

Q: "How would you make this real-time?"

A: Add a streaming pipeline (Kafka/PubSub) for new documents. Incrementally update FAISS index using add instead of rebuild. Use a write-ahead log for durability. Background worker handles re-encoding and index optimization.

Q: "How would you handle multi-tenant search?"

A: Add tenant_id to document metadata. Filter FAISS results by tenant prefix. Or use separate indices per tenant for isolation. Cache tenant-specific indices.

Q: "How would you add personalization?"

A: Track user click history and dwell time. Build user embedding from interaction history. Re-rank results using user similarity. Implement explore/exploit trade-off with MAB.

ML Questions

Q: "Why all-MiniLM-L6-v2?"

A: Best quality-to-size ratio: 384 dimensions, 80MB, runs on CPU. Larger models (all-mpnet-base-v2) give ~2% better quality but 4x slower. For search, latency matters.

Q: "How do you handle out-of-vocabulary words?"

A: Sentence-BERT uses WordPiece tokenization, which handles most OOV via subword units. For rare technical terms, spell correction and query expansion help.

Q: "How would you improve search quality?"

A: (1) Domain-specific fine-tuning of embeddings, (2) Learning-to-rank with click data, (3) Query reformulation with LLMs, (4) Personalization, (5) Freshness signals.

14. Resources & References

Learning Resources

- FAISS Documentation: https://faiss.ai/
- Sentence-Transformers: https://www.sbert.net/
- BM25 Paper: https://en.wikipedia.org/wiki/Okapi_BM25
- Reciprocal Rank Fusion: https://dl.acm.org/doi/10.1145/2926867
- FastAPI Documentation: https://fastapi.tiangolo.com/
- Streamlit Documentation: https://docs.streamlit.io/

Datasets

- Wikipedia: HuggingFace datasets - wikipedia (200K+ articles)
- Evaluation: TREC dataset or create custom labeled queries
- Alternative: MS MARCO Passage Ranking (13M passages)

Free Tools

- Google Colab: Free GPU for training embeddings
- Streamlit Cloud: Free hosting for ML apps
- HuggingFace Spaces: Alternative free hosting
- GitHub: Free repository hosting with Actions CI
- Weights & Biases: Free experiment tracking (personal tier)

This project demonstrates: ML engineering, systems thinking, API design, frontend skills, evaluation rigor, and deployment - exactly what MAANG/FAANG interviewers look for in final-year projects. Good luck!